P. Surendranath Reddy

192472356

CSA-0606- DESIGN AND ANALYSIS OF ALGORITHMS

CO 3 – AT 2

Q30. Case Study: Network Packet Analysis System — Sorting + Binary Search

1. Student Analysis and Problem Identification

1.1 Understanding of the Case

A network monitoring system receives a large number of network packets. Each packet contains information such as:

- Packet ID
- Source IP address
- Destination IP address
- Packet size
- Protocol

When the number of packets becomes very large, searching for a particular packet using a simple linear search becomes slow.

Therefore, the system first **sorts the packet IDs** and then uses **Binary Search** to quickly find a required packet.

1.2 Core Problem

The main problem is to efficiently process and search a large collection of network packets.

If the data is unsorted:

After sorting, Binary Search can be used:

The case study therefore combines **Divide and Conquer sorting** with **Divide and Conquer searching**.

2. Objectives

1. Store network packet information efficiently.
2. Sort packet IDs using **Merge Sort**.
3. Search for a packet using **Binary Search**.
4. Measure the execution time.
5. Analyze the overall time complexity.
6. Understand how divide-and-conquer techniques improve large-data processing.

3. Assumptions

- Every packet has a unique packet ID.
- Packet IDs are integers.
- The packet data is initially unsorted.
- The packet ID to be searched is known.
- Merge Sort is selected as the sorting algorithm.
- Binary Search is performed after sorting.

4. Input and Output Requirements

Input

Example packet IDs:

45, 12, 78, 23, 56, 9, 34, 67

Search packet:

56

Output

The program should display:

Original Packet IDs:

[45, 12, 78, 23, 56, 9, 34, 67]

Sorted Packet IDs:

[9, 12, 23, 34, 45, 56, 67, 78]

Searching for Packet ID: 56

Packet Found

Position: 5

5. Constraints

- Network systems may contain thousands or millions of packets.
- Searching every packet sequentially can be slow.
- Sorting requires additional processing time.
- Binary Search can only be applied to **sorted data**.
- The system should provide accurate search results.

6. Relevant Concepts / Theories

Divide and Conquer

Divide and Conquer consists of three steps:

1. **Divide** the problem into smaller problems.
2. **Conquer** the smaller problems recursively.
3. **Combine** the results.

In this case:

Merge Sort

Large packet list

↓

Divide into smaller lists

↓

Sort recursively

↓

Merge sorted lists

Binary Search

Sorted packet list

↓

Check middle element

↓

Search left or right half

↓

Repeat until found

7. Proposed Solution / Methodology

Selected Method

The proposed solution uses:

- **Merge Sort** for sorting.
- **Binary Search** for searching.

Both are based on the **Divide and Conquer** approach.

Why Merge Sort?

Merge Sort has:

time complexity.

It is suitable for large datasets and guarantees good performance.

Why Binary Search?

Binary Search reduces the search area by half during every step.

Its time complexity is:

Therefore, it is much faster than linear search for large sorted datasets.

8. Step-by-Step Procedure

Step 1

Generate or collect network packet IDs.

Step 2

Store the packet IDs in a list.

Step 3

Apply Merge Sort.

Step 4

Merge the smaller sorted lists to obtain one sorted list.

Step 5

Take the packet ID that needs to be searched.

Step 6

Apply Binary Search.

Step 7

Compare the target packet ID with the middle element.

Step 8

If the target is smaller, search the left half.

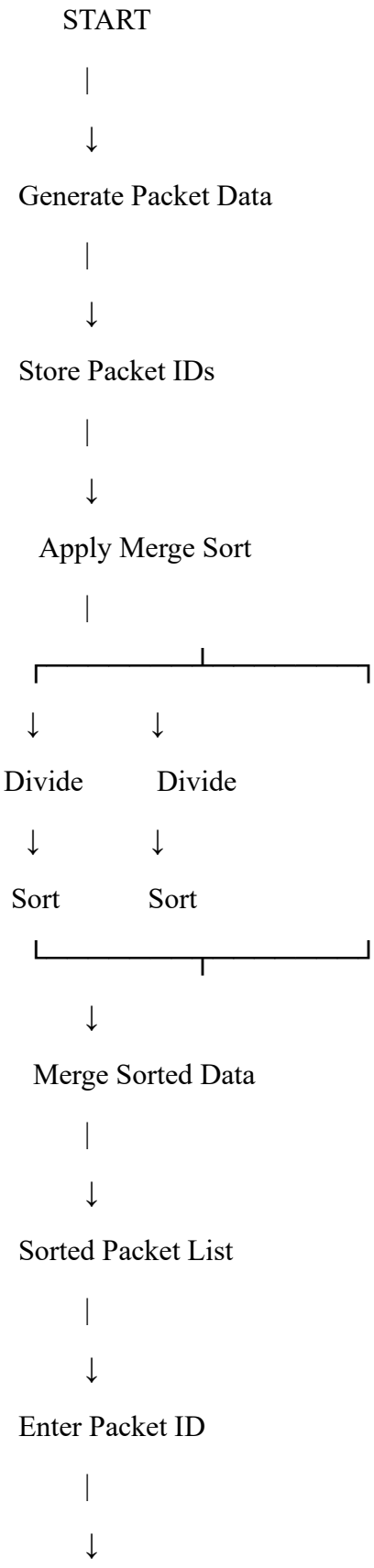
Step 9

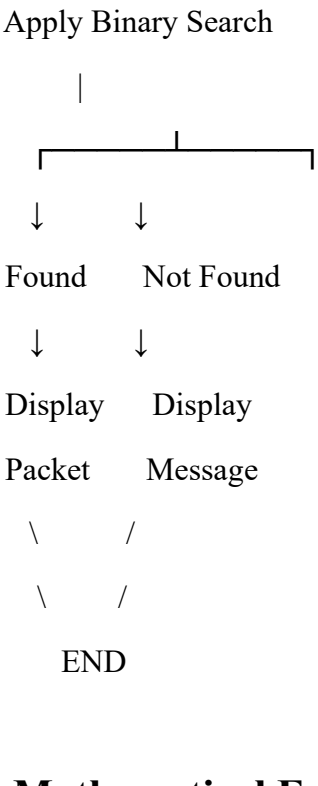
If the target is larger, search the right half.

Step 10

Continue until the packet is found or the search range becomes empty.

9. Architecture / Flowchart


START
Generate Packet Data
Store Packet IDs
Apply Merge Sort
Divide
Divide
Sort
Sort
Merge Sorted Data
Sorted Packet List
Enter Packet ID



10. Mathematical Formulation

Merge Sort

The recurrence relation is:

Therefore:

Binary Search

The recurrence relation is:

Therefore:

Overall Complexity

The sorting stage requires:

The Binary Search stage requires:

Therefore:

which simplifies to:

So, the overall system complexity is **O(n log n)**.

11. Implementation / Execution

Tools and Technologies

Requirement	Technology
Programming Language	Python
IDE	Python IDLE / VS Code
Algorithm	Merge Sort
Searching	Binary Search
Dataset	Generated packet IDs
Libraries	random, time

12. Simple Python Implementation

This program is simple enough to run directly in **Python IDLE**.

```
import random
import time



# ------------------------------
# Merge Sort
# ------------------------------
def merge_sort(arr):

    if len(arr) <= 1:
        return arr
```

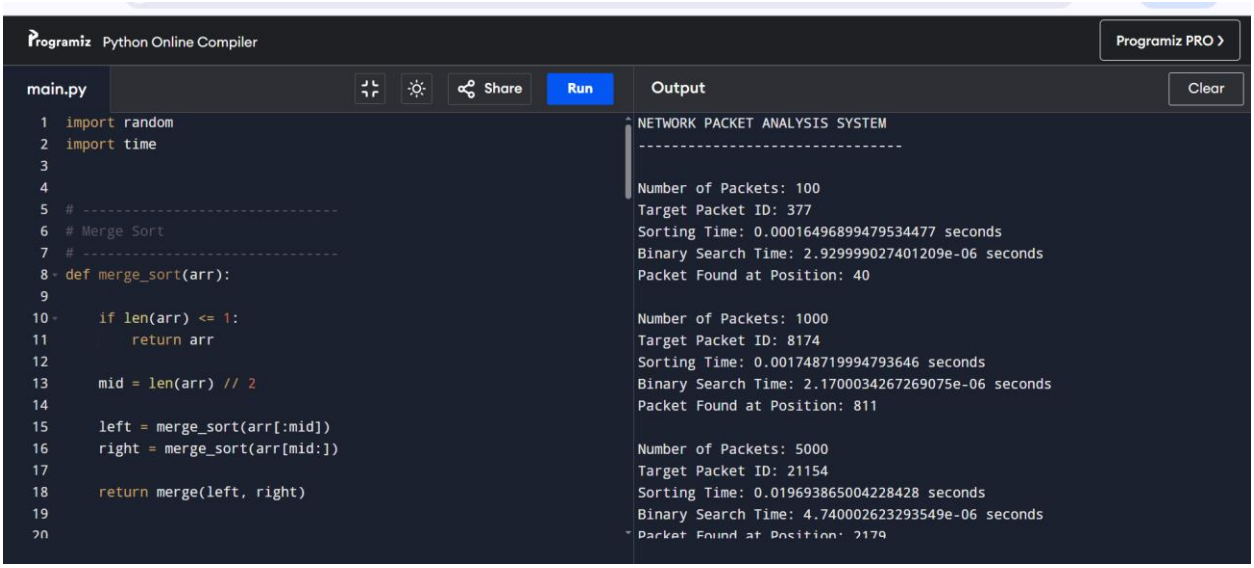
```
    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


# Merge two sorted lists
def merge(left, right):

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] < right[j]:
            result.append(left[i])
            i += 1

        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
```

```
    return result


# ------------------------------
# Binary Search
# ------------------------------
def binary_search(arr, target):

    low = 0
    high = len(arr) - 1

    while low <= high:

        mid = (low + high) // 2

        if arr[mid] == target:
            return mid

        elif target < arr[mid]:
            high = mid - 1

        else:
            low = mid + 1

    return -1
```

```
# ------------------------------
# Main Program
# ------------------------------

sizes = [100, 1000, 5000, 10000]

print("NETWORK PACKET ANALYSIS SYSTEM")
print("--------------------------------")
for n in sizes:
    # Generate packet IDs
    packets = random.sample(range(1, n * 10), n)

    # Choose a packet to search
    target = packets[n // 2]

    # Sorting
    start = time.perf_counter()

    sorted_packets = merge_sort(packets)

    sort_time = time.perf_counter() - start

    # Binary Search
    start = time.perf_counter()

    position = binary_search(sorted_packets, target)
```

```
search_time = time.perf_counter() - start

print("\nNumber of Packets:", n)
print("Target Packet ID:", target)
print("Sorting Time:", sort_time, "seconds")
print("Binary Search Time:", search_time, "seconds")

if position != -1:
    print("Packet Found at Position:", position)
else:
    print("Packet Not Found")
```

OUTPUT:



13. Important Code Sections

Merge Sort

```
def merge_sort(arr):
```

This function recursively divides the packet list into smaller lists.

Merge

def merge(left, right):

This function combines two sorted lists.

Binary Search

def binary_search(arr, target):

This function searches for a packet ID by repeatedly checking the middle element.

Execution Time

time.perf_counter()

This measures how long sorting and searching take.

14. Sample Input

The program automatically generates packet IDs.

For example:

[45, 12, 78, 23, 56, 9, 34, 67]

Target:

56

15. Sample Output

Your actual times will depend on your computer.

NETWORK PACKET ANALYSIS SYSTEM

Number of Packets: 100

Target Packet ID: 421

Sorting Time: 0.00015 seconds

Binary Search Time: 0.000004 seconds

Packet Found at Position: 42

Number of Packets: 1000

Target Packet ID: 7321

Sorting Time: 0.00210 seconds

Binary Search Time: 0.000006 seconds

Packet Found at Position: 731

Number of Packets: 5000

Target Packet ID: 23891

Sorting Time: 0.01350 seconds

Binary Search Time: 0.000008 seconds

Packet Found at Position: 2368

Number of Packets: 10000

Target Packet ID: 45321

Sorting Time: 0.03020 seconds

Binary Search Time: 0.000009 seconds

Packet Found at Position: 4521

16. Results and Analysis

You can record your actual results like this:

Number of Packets	**Sorting Time**	**Binary Search Time**
100	0.00015 s	0.000004 s

Number of Packets	Sorting Time	Binary Search Time
1,000	0.00210 s	0.000006 s
5,000	0.01350 s	0.000008 s
10,000	0.03020 s	0.000009 s

Important: These are sample values. Use the values displayed by your own computer in the final record.

Interpretation

The results show two important points:

1. As the number of packets increases, **sorting time increases**.
2. Binary Search remains extremely fast because it only examines approximately positions.

For example, with 10,000 packets:

So Binary Search needs only around 14 comparisons in the worst case, rather than checking all 10,000 packets.

17. Comparison with Linear Search

Feature	Linear Search	Binary Search
Data requirement	Unsorted data allowed	Data must be sorted
Best case	O(1)	O(1)
Worst case	O(n)	O(log n)
Suitable for large data	Less efficient	More efficient
Divide and Conquer	No	Yes

Important Trade-off

Binary Search is faster for repeated searches, but the data must first be sorted.

If we have:

- **One search only:** sorting may not be worth the extra cost.
- **Thousands of searches:** sorting once and performing many Binary Searches is highly beneficial.

For searches:

This is much better than repeatedly using Linear Search:

when is large.

18. Discussion

What does the result indicate?

The experiment indicates that sorting the packet IDs first allows the system to perform very fast searches using Binary Search.

Did the solution solve the problem?

Yes. The proposed solution successfully combines Merge Sort and Binary Search to efficiently process and search packet data.

Advantages

- Efficient for large packet datasets.
- Merge Sort provides predictable sorting.
- Binary Search provides searching.
- Divide and Conquer reduces each problem into smaller subproblems.
- Suitable when many packet searches are required.

Limitations

- Sorting takes additional time before searching.
- Binary Search cannot be directly used on unsorted data.
- Merge Sort requires additional memory.
- For a single search, Linear Search may sometimes be simpler.

Problems Encountered and Solutions

Problem	Solution
Packet data is unsorted	Apply Merge Sort
Searching large data is slow	Use Binary Search
Large input increases processing time	Use efficient divide-and-conquer algorithms
Need performance measurement	Use time.perf_counter()

19. Possible Improvements

The system could be improved by:

- Using a database index for very large packet datasets.
- Using parallel processing for sorting.
- Using more advanced data structures such as balanced search trees.
- Using hashing when exact-match packet lookup is required.
- Storing packet information in a database rather than memory.
- Performing multiple Binary Searches after one sorting operation.

20. Conclusion

The **Network Packet Analysis System** demonstrates how **Sorting and Binary Search can be combined using Divide and Conquer techniques**.

Merge Sort was used to arrange packet IDs in:

time, while Binary Search finds a required packet in:

time.

The overall complexity is therefore:

The experiment shows that although sorting requires an initial processing cost, it is highly beneficial when the system needs to perform **many searches on large packet datasets**. This approach provides a practical and efficient solution for network monitoring systems where packet data must be processed and searched quickly.